

Device Integrity Risk Scorer : Plateforme d'Audit Défensif pour Android — Détection de Signaux de Compromission

Houda Oubane^a, Niaama Naftaoui^a, Fatima-Ezzahra Kasraoui^a,
Fatima-Ezzahra El Boudhiri^a

^a*École Nationale des Sciences Appliquées de Marrakech, Université Cadi Ayyad,
Marrakech, Maroc*

Abstract

Face à la prolifération des appareils Android modifiés, des frameworks de hooking dynamiques et des techniques de contournement avancées telles que Magisk, Frida et LSPosed, la nécessité d'outils d'audit défensifs automatisés devient critique pour les développeurs et les équipes de sécurité mobile. Ce rapport présente Device Integrity Risk Scorer, une plateforme d'audit passif qui analyse l'environnement d'exécution Android et produit un score de risque quantifié entre 0 et 100. Contrairement aux approches classiques qui se limitent à bloquer l'exécution, cet outil adopte un mode d'observation non-intrusif : il collecte des signaux techniques mesurables (`ro.debuggable`, présence de `su`/Magisk, activation de Frida/LSPosed, mode SELinux, intégrité des binaires système, permissions dangereuses) via ADB et les agrège grâce à un moteur de scoring pondéré doté d'un mécanisme de réduction des faux positifs. Développée en Python et packagée comme outil CLI installable, la plateforme génère des rapports lisibles en console et des exports JSON exploitables dans des pipelines CI/CD. Ce rapport documente l'architecture, les fonctionnalités, le processus technique détaillé, ainsi que cinq scénarios de test réels couvrant des environnements allant d'un téléphone physique non rooté à un émulateur avec Magisk, LSPosed, Frida et SELinux en mode Permissive.

Keywords: Sécurité Android, Audit Défensif, Détection de Root, Frida, Magisk, LSPosed, SELinux, ADB, Score de Risque, CI/CD Mobile

Required Metadata

Current code version

Ancillary data table required for subversion of the codebase. Kindly replace examples in the right column with the correct information about your current code, and leave the left column as it is.

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.0.0
C2	Permanent GitHub link to code/repository used for this code version	Non disponible (projet pédagogique Lab 9)
C3	Legal Code License	Usage pédagogique — Lab 9
C4	Code versioning system used	Git
C5	Software code languages, tools, and services used	Python 3.9+, ADB, Drozer, Frida CLI
C6	Compilation requirements, operating environments & dependencies	Python 3.9+, ADB Platform Tools, drozer, frida-tools, appareil Android 10+ (physique ou émulateur)
C7	If available link to developer documentation/manual	<code>setup.py</code> , <code>requirements.txt</code> (inclus dans le référentiel)
C8	Support email for questions	(non communiqué)

TABLE 1 – Code metadata (mandatory)

1. Motivation et Signification

1.1. Contexte et Problématique

Le développement d'applications mobiles Android s'accompagne d'une menace croissante liée à l'exécution sur des appareils modifiés. Un appareil rooté, instrumenté par Frida, ou doté d'un module LSPosed, représente un environnement d'exécution fondamentalement non fiable pour toute application sensible — bancaire, médicale, gouvernementale ou d'entreprise. Les attaquants exploitent ces environnements pour intercepter des appels de méthodes Java, manipuler la logique applicative, extraire des secrets cryptographiques ou contourner des mécanismes d'authentification.

Les frameworks de root modernes comme Magisk compliquent considérablement la détection côté application : ils opèrent au niveau du bootloader, masquent leur présence aux vérifications classiques (SafetyNet, Play Integrity API) et permettent d'installer des modules d'hook permanents comme LSPosed, successeur d'Xposed Framework. Parallèlement, Frida permet une

instrumentation dynamique non persistante mais extrêmement puissante, capable de hooker n'importe quelle méthode Java, Kotlin ou native à l'exécution.

Face à cette réalité, les équipes de sécurité mobile ont besoin d'outils d'audit qui permettent d'évaluer objectivement le niveau de risque d'un environnement Android avant tout déploiement, test ou audit de sécurité. Ces outils doivent opérer en mode passif — sans modifier l'appareil — et produire des résultats reproductibles, exploitables et intégrables dans des workflows automatisés.

1.2. Lacunes des Solutions Existantes

Les approches actuelles souffrent de plusieurs limitations majeures :

- Les solutions côté application (SafetyNet, Play Integrity API) nécessitent une infrastructure serveur Google, une connexion réseau, et peuvent être contournées par des modules Magisk spécialisés (MagiskHide, Shamiko).
- Les outils de pentest mobiles (Drozer, MobSF) sont orientés vers l'analyse offensive des applications et non vers l'audit de l'environnement d'exécution.
- Les scripts ADB manuels exigent une expertise technique élevée, ne produisent pas de score agrégé, et ne sont pas reproductibles dans un contexte CI/CD.
- L'absence de mécanisme de réduction des faux positifs dans les outils existants entraîne des scores élevés sur des ROMs custom légitimes (OnePlus, Xiaomi) où le root est activé par le fabricant à des fins de développement.

1.3. Contribution du Projet

Le Device Integrity Risk Scorer comble ces lacunes en proposant :

1. Un outil d'audit passif autonome qui n'exige aucune infrastructure cloud et fonctionne hors ligne via ADB.
2. Un score quantifié de 0 à 100 avec niveaux de risque (FAIBLE / MOYEN / ÉLEVÉ) permettant une interprétation immédiate.
3. Un moteur de règles pondérées qui reflète la dangerosité réelle de chaque signal — Frida actif est plus dangereux que le seul mode debug.
4. Un mécanisme de réduction des faux positifs (FP Reducer) qui contextualise les signaux selon le modèle d'appareil.
5. Une intégration CI/CD via export JSON, permettant d'automatiser les vérifications d'intégrité dans les pipelines de test mobile.

6. Une interface CLI élaborée avec un rapport coloré, des explications pédagogiques pour chaque signal, et des commandes de remédiation directement exécutables.

2. Description du Logiciel

2.1. Architecture du Logiciel

L'architecture du Device Integrity Risk Scorer est organisée en trois couches fonctionnelles distinctes, communiquant de manière séquentielle selon le pipeline Collecte → Scoring → Rapport.

Couche 1 — Collecte (collect/) : Cette couche est responsable de la communication avec l'appareil Android via ADB. Le module `drozer_collect.py` exécute des commandes shell ADB à l'aide du sous-processus Python et récupère les valeurs des propriétés système (`getprop`), l'état des processus (`ps -A`), la liste des packages installés (`pm list packages`), et les métadonnées de sécurité (`getenforce`). La collecte produit un dictionnaire Python structuré qui est sérialisé en JSON dans `scorer/signals.json`.

Couche 2 — Scoring (scorer/) : Cette couche contient le moteur d'évaluation des risques. Elle se compose de deux modules complémentaires : `score_engine.py` qui calcule le score brut par somme pondérée des signaux détectés, et `fp_reducer.py` qui applique des règles de correction pour réduire les faux positifs liés aux environnements légitimes. Le score final est plafonné à 100 et classifié selon trois niveaux de risque.

Couche 3 — Rapport (report/) : Cette couche gère la présentation des résultats. Le module `audit.py` (point d'entrée principal) produit un rapport console riche avec colorisation ANSI, explications pédagogiques pour chaque signal, barre de progression visuelle et recommandations de remédiation dynamiques. Le module `json_report.py` exporte simultanément un rapport structuré en JSON pour une intégration dans des pipelines CI/CD.

2.2. Fonctionnalités du Logiciel

Le Device Integrity Risk Scorer offre un ensemble de fonctionnalités couvrant l'ensemble du cycle d'audit défensif Android.

1. Collecte Automatisée des Signaux de Sécurité — Le module de collecte automatise l'interrogation de neuf catégories de signaux distincts via ADB. Il vérifie la disponibilité de l'appareil avant toute collecte (`adb_available()`), gère les timeouts et exceptions pour les commandes ADB qui échouent ou prennent trop de temps, et produit un dictionnaire structuré avec horodatage, modèle de l'appareil et version Android. Cette collecte prend typiquement 5 à 15 secondes selon la réactivité de l'appareil.

2. Moteur de Scoring Pondéré — Chaque signal détecté contribue au score total selon un poids qui reflète sa dangerosité réelle dans un contexte d’attaque. La présence du binaire `su` (root traditionnel) est plus grave que le simple mode debug activé, car elle indique un accès privilégié direct au système. Le moteur agrège les poids des signaux détectés et plafonne le résultat à 100 pour maintenir une échelle normalisée.

3. Réduction des Faux Positifs Contextualisée — Le module `fp_reducer.py` applique deux règles de correction essentielles. La première règle identifie les appareils dont le constructeur active le root nativement (OnePlus, Xiaomi, Poco, Redmi) sans présence de hooks actifs, et réduit le score de 25 points. La deuxième règle identifie les environnements de développement légitime où seul le mode debug est activé (sans root ni hook), et réduit le score de 15 points.

4. Rapport Console Pédagogique et Colorisé — Pour chaque signal, le rapport fournit : le statut (DANGER / SÉCURISÉ / WARNING), une explication en langage naturel du risque associé, et le risque spécifique pour l’application. Cette approche pédagogique permet à un développeur non spécialiste de comprendre immédiatement ce que signifie chaque signal et pourquoi il est dangereux.

5. Recommandations de Remédiation Dynamiques — Les recommandations affichées sont générées dynamiquement selon les signaux effectivement détectés. Si Frida est détecté, la recommandation spécifique à Frida est affichée. Si SELinux est en mode Permissive, la commande de remédiation exacte (`adb shell setenforce 1`) est fournie directement dans le terminal.

6. Export JSON pour CI/CD — Le rapport JSON généré dans `report/audit_report.json` contient l’ensemble des données structurées : horodatage, modèle, version Android, score, niveau de risque, état de chaque signal, et liste des recommandations. Ce format permet d’intégrer l’audit dans des pipelines GitHub Actions, Jenkins ou GitLab CI.

7. Détection des Permissions Dangereuses — En complément des signaux système, l’outil analyse les permissions Android accordées aux applications installées sur l’appareil. Il vérifie la présence de huit permissions critiques : `READ_CONTACTS`, `ACCESS_FINE_LOCATION`, `RECORD_AUDIO`, `CAMERA`, `READ_SMS`, `SEND_SMS`, `READ_CALL_LOG`, et `ACCESS_BACKGROUND_LOCATION`.

2.3. Processus Technique — Pipeline de Détection

2.3.1. Phase 1 : Collecte des Signaux via ADB

Signal 1 — Mode Debug Android (`ro.debuggable`) : La propriété `ro.debuggable` est inscrite dans le noyau Linux au moment du boot. Sur un appareil de production, elle vaut toujours 0. Sur un émulateur ou un

build de développement, elle vaut 1 — ce qui autorise n’importe quelle application à se connecter au processus de débogage d’une autre application via JDWP.

Signal 2 — Signature du Firmware (build tags) : Les firmwares officiels sont signés avec des clés de production (**release-keys**). Un firmware signé avec des **test-keys** indique soit une ROM custom (LineageOS, PixelExperience), soit un build de développement interne, soit — dans le pire cas — un firmware modifié malicieusement.

Signal 3 — Binaire su (root traditionnel) : Le binaire **su** (substitute user) est le mécanisme classique d’élévation de privilèges sous Linux/Android. Sa présence signifie qu’une application peut demander les droits root, contournant complètement le sandboxing par UID d’Android.

Signal 4 — Framework Magisk (root moderne) : Magisk opère différemment du root traditionnel : il crée un espace de noms de montage isolé qui masque les modifications au système. L’outil le détecte en vérifiant l’existence du répertoire de travail de Magisk (`/data/adb/magisk/`) et la présence de son package (`com.topjohnwu.magisk`).

Signal 5 — Frida (hook dynamique) : Frida-server est un processus qui tourne en arrière-plan sur l’appareil cible et permet au client Frida sur le PC de s’y connecter pour injecter des scripts JavaScript dans n’importe quel processus. Sa détection par `ps -A` est efficace lorsque le serveur n’a pas été renommé.

Signal 6 — LSPosed / Xposed (hook permanent) : LSPosed est le successeur d’Xposed Framework, compatible avec les versions Android récentes. Contrairement à Frida qui est temporaire, LSPosed modifie le runtime Android de façon permanente et survit aux redémarrages. Il est typiquement installé via un module Magisk.

Signal 7 — SELinux (protection MAC) : SELinux (Security-Enhanced Linux) est le mécanisme de contrôle d’accès obligatoire d’Android. En mode Enforcing, il applique les politiques de sécurité et bloque les actions non autorisées. En mode Permissive, il log les violations mais ne bloque rien — protection nulle.

Signal 8 — Intégrité des Binaires Système : Un binaire système modifié (checksum différent d’une baseline de référence) indique un tampering au niveau du système de fichiers.

Signal 9 — Permissions Dangereuses : Vérification des permissions critiques dans les applications installées.

Signal	Points	Justification
<code>ro.debuggable = 1</code>	+15	Debug actif — application déboguable
<code>ro.build.tags = test-keys</code>	+10	Firmware non officiel
Binaire <code>su</code> trouvé	+20	Accès root, bypass sandbox
Magisk paquet/dossier	+15	Root moderne, persistant
Frida-server actif	+15	Hooking dynamique
LSPosed/Xposed	+10	Modification permanente du runtime
SELinux Permissive	+5	Protection MAC désactivée
Binaires système modifiés	+5	Tampering confirmé
Permissions dangereuses	+5	Exposition des données

TABLE 2 – Poids des signaux de détection

2.3.2. Phase 2 : Calcul du Score Pondéré

2.3.3. Phase 3 : Réduction des Faux Positifs

Règle 1 — ROM Custom Légitime : Si le modèle de l'appareil appartient à une marque connue pour activer le root par défaut (OnePlus, Xiaomi, Poco, Redmi) *et* qu'aucun hook actif (Frida ou Xposed) n'est détecté, le score est réduit de 25 points.

Règle 2 — Environnement de Développement Debug-Only : Si seul le mode debug est activé, sans aucun indicateur de root (`su`, Magisk) ni de hook (Frida, Xposed), le score est réduit de 15 points.

2.3.4. Phase 4 : Niveaux de Risque et Classification

Score	Niveau	Signification	Action recommandée
0–39	FAIBLE ★○○○	Appareil sain	Bonnes pratiques
40–69	MOYEN ★★○	Signaux suspects	Audit approfondi
70–100	ÉLEVÉ ★★★	Compromis	Play Integrity API

TABLE 3 – Niveaux de risque et classification

3. Exemples Illustratifs — Scénarios de Test Réels

3.1. Contexte et Environnement de Test

Pour valider l'outil dans des conditions représentatives, deux environnements distincts ont été utilisés :

- **Appareil 1** : Émulateur Genymotion — environnement simulé, rooté.
- **Appareil 2** : Téléphone physique réel (OPPO CPH2239, Android 11) — appareil de production standard.

L'émulateur a été progressivement modifié pour simuler différents niveaux de compromission, depuis un état rooté basique jusqu'à un état avec Frida actif, Magisk installé et LSPosed activé. Le téléphone physique représente l'environnement « sain » de référence.

3.2. Scénario 1 — Émulateur avec Magisk et LSPosed Installés (Score ÉLEVÉ)

Objectif : Simuler un appareil avec une infrastructure de root avancée et persistante (Magisk + LSPosed), représentant le profil d'un appareil utilisé pour du reverse engineering ou de la manipulation d'applications mobiles.



FIGURE 1 — Scénario 1 — Exécution de l'outil et analyse des signaux de sécurité sur l'émulateur (Magisk + LSPosed installés).

Signaux détectés et analyse des résultats :

- **Mode Debug Android** : DANGER (ro.debuggable = 1, +15 points). L'émulateur est en mode debug.
- **Signature du firmware** : DANGER (build tags = test-keys, +10 points). Firmware non officiel signé avec des clés de test.
- **Binaire su** : DANGER (su trouvé, +20 points). Accès root direct.
- **Framework Magisk** : DANGER (+15 points). Root moderne détecté, persistant.
- **Frida** : SÉCURISÉ — Frida non détecté.

- **XPosed/LSPosed** : DANGER (+10 points). Modification permanente du runtime Android.
- **SELinux** : WARNING — mode Permissive (+5 points).
- **Intégrité des binaires** : SÉCURISÉ.
- **Permissions dangereuses** : DANGER (+5 points).



FIGURE 2 – Scénario 1 — Score final 80/100 [ÉLEVÉ] et commandes de remédiation générées dynamiquement.

Score obtenu : 80/100 [ÉLEVÉ]. Ce scénario représente le profil type d'un appareil de chercheur en sécurité ou d'un attaquant sophistiqué : root persistant via Magisk, hooks permanents via LSPosed, et SELinux désactivé.

3.3. Scénario 2 — Émulateur avec Frida et LSPosed Actifs (Score Maximum)

Objectif : Valider que l'outil détecte simultanément la présence de Frida-server actif et de XPosed/LSPosed, reflétant le niveau de risque maximal d'un environnement entièrement compromis.

```

[5] Frida (hook framework)
  DANGER : Frida actif
  -> Explication : Frida permet d'intercepter et modifier les
      appels de methodes Java/Python a l'execution.
  -> Risque : Espionnage des methodes, contournement de securites

[6] XPosed / LSPosed (hook permanent)
  DANGER : XPosed detecte
  -> Explication : XPosed modifie le runtime Android de facon
      permanente pour intercepter les appels systeme.
  -> Risque : Modification permanente, contournement generalise

[7] SELinux (protection MAC)
  WARNING : SELinux en mode Permissive
  -> Explication : SELinux en mode Permissive n'applique aucune
      politique de securite. Toutes les actions sont autorisees.
  -> Risque : Protection desactivee, aucun controle d'accès

[8] Integrite des binaires systeme
  SECURISE : Binaires integres
  -> Statut : CONFORME

[9] Permissions dangereuses
  DANGER : 8 permission(s) dangereuse(s) detectee(s)
  -> Explication : Des applications peuvent acceder a vos donnees sensibles
  -> Permissions : READ_CONTACTS, ACCESS_FINE_LOCATION, RECORD_AUDIO, CAMERA, READ_SMS, SEND_SMS, READ_CALL_LOG, ACCESS_BACKGROUND_LOCA
      ATION
  -> Risque : Exposition des donnees personnelles, vie privee

=====
SCORE DE RISQUE
=====

SCORE FINAL : 95/100 [ELEVÉ] ***
[Progress bar: 95% full]
=====

```

FIGURE 3 – Scénario 2 — Signaux détectés : [5] Frida DANGER (+15 pts), [6] XPosed DANGER (+10 pts), [7] SELinux WARNING, [9] Permissions DANGER. Score final : **95/100 [ÉLEVÉ]** — niveau de risque maximal, toutes les couches de compromission actives.

Résultat clé : L’activation simultanée de Frida-server et de XPosed/LS-Posed représente la combinaison la plus dangereuse : hooking dynamique en temps réel (Frida, +15 pts) et modification permanente du runtime (LS-Posed, +10 pts). Toutes les méthodes Java, Kotlin et natives peuvent être interceptées et modifiées. Le score 95/100 confirme que le moteur de scoring discrimine correctement ce niveau d’exposition maximal.

3.4. Scénario 3 — Téléphone Physique Non Rooté (Score FAIBLE)

Objectif : Valider que l’outil produit un score minimal sur un appareil de production standard non modifié (OPPO CPH2239, Android 11).

```
PS C:\Users\hp\Downloads\butterfly-audit-frame-main\butterfly-audit-frame-main> risk-scorer -audit
-> Statut : CONFORME

[8] Integrite des binaires systeme
SECURISE : Binaires integres
-> Statut : CONFORME

[9] Permissions dangereuses
DANGER : 8 permission(s) dangereuse(s) detectee(s)
-> Explication : Des applications peuvent acceder a vos donnees sensibles
-> Permissions : READ_CONTACTS, ACCESS_FINE_LOCATION, RECORD_AUDIO, CAMERA, READ_SMS, SEND_SMS, READ_CALL_LOG, ACCESS_BACKGROUND_LOCATION
N
-> Risque : Exposition des donnees personnelles, vie privée

=====
SCORE DE RISQUE
=====

SCORE FINAL : 5/100 [FAIBLE] ★☆☆
[ ]

=====
RECOMMANDATIONS
=====

Actions correctives :

1. Permissions dangereuses (8) : Verifier les applications ayant acces aux donnees sensibles

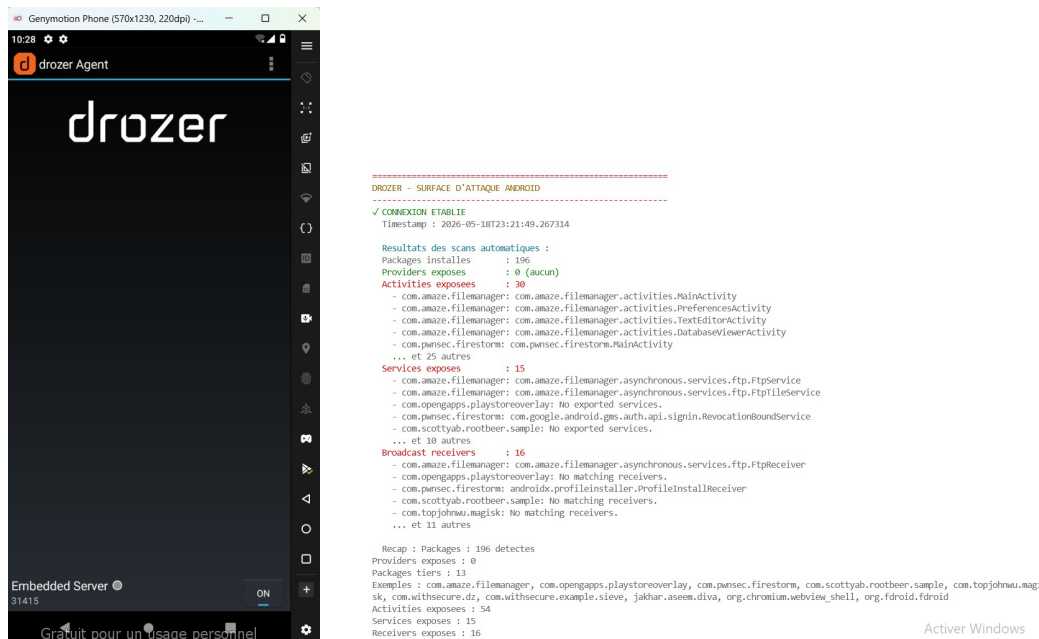
=====
Commandes rapides :
=====
```

FIGURE 4 – Scénario 3 — Audit du téléphone physique OPPO CPH2239 via `risk-scorer -audit` : seul le signal [9] Permissions est en DANGER (+5 pts). Score final : **5/100 [FAIBLE]** — appareil sain en production. Recommandation unique : vérifier les applications ayant accès aux données sensibles.

Score obtenu : 5/100 [FAIBLE]. Ce résultat valide la cohérence de l’outil : un téléphone physique non rooté de production obtient le score le plus bas possible. La seule recommandation affichée est : « Permissions dangereuses (8) : Vérifier les applications ayant accès aux données sensibles » — une recommandation de vigilance, non une alerte critique.

3.5. Fonctionnalité Étendue — Intégration Drozer (*-with-drozer*)

En complément des scénarios présentés, l’outil offre un mode étendu activé via le flag `-with-drozer`, qui intègre une analyse automatisée de la surface d’attaque Android directement dans le pipeline d’audit.



(a) Interface Drozer Agent sur l'émulateur Genymotion : serveur embarqué actif (port 31415, *Embedded Server* ON).

(b) Résultats Drozer — Surface d'attaque Android : 196 packages (13 tiers), 30 activités exposées, 15 services et 16 broadcast receivers détectés.

FIGURE 5 – Fonctionnalité **-with-drozer** : agent Drozer actif sur l'émulateur (gauche) et résultats de l'analyse automatisée de la surface d'attaque (droite).

L'analyse Drozer sur l'émulateur recense 196 packages installés dont 13 tiers, avec une surface d'attaque limitée (30 activités exposées, 15 services, 16 broadcast receivers), reflétant la nature minimale d'un environnement de test. Sur un téléphone physique de production, cette surface est significativement plus large : un appareil réel typique présente plusieurs centaines d'activités exposées et de nombreux services tiers, illustrant pourquoi l'audit sur émulateur seul est insuffisant.

4. Impact et Discussion

4.1. Pertinence des Résultats de Test

Ce gradient de scores valide que le moteur de scoring distingue correctement les niveaux de compromission et que l'ajout de chaque couche de menace est bien reflété dans le score final.

4.2. Limitation Connue : Le Hash de Référence des Binaires

La limitation la plus significative identifiée lors des tests concerne le signal d'intégrité des binaires système (Signal 8). L'outil utilise un hash MD5 fixe

Scénario	Score	Niveau	Signaux clés
1 — Magisk + LSPosed	80/100	ÉLEVÉ	Debug, test-keys, su, Magisk, XPosed
2 — Frida + LSPosed actifs	95/100	ÉLEVÉ	+ Frida actif
3 — Téléphone physique	5/100	FAIBLE	Permissions uniquement

TABLE 4 – Résumé des résultats de test

comme référence pour `/system/bin/sh`. Sur tout autre appareil — physique ou émulateur différent — ce hash ne correspond pas, entraînant un faux positif systématique de 5 points.

Cette limitation est inhérente à l’approche par hash fixe. Une solution robuste nécessiterait une base de données de hashes de référence par modèle, version Android et build, ou l’utilisation d’une API comme Android Verified Boot pour vérifier l’intégrité des partitions système.

4.3. Valeur Ajoutée par Rapport aux Outils Existants

- **Par rapport aux scripts ADB manuels** : L’agrégation automatique de 9 signaux en un score unique, les explications pédagogiques et les recommandations dynamiques réduisent considérablement le temps d’audit et le niveau d’expertise requis.
- **Par rapport à Drozer seul** : Drozer analyse la surface d’attaque des applications (composants exposés, IPC), mais ne produit pas d’évaluation de l’environnement d’exécution. Les deux outils sont complémentaires.
- **Par rapport à SafetyNet/Play Integrity API** : Ces APIs nécessitent une connexion réseau aux serveurs Google et peuvent être contournées par MagiskHide/Shamiko. L’outil fonctionne hors ligne depuis le PC et n’est pas contournable depuis l’appareil audité.
- **Par rapport à MobSF** : MobSF est orienté analyse statique/dynamique d’APK, non audit de l’environnement d’exécution système.

4.4. Intégration dans un Pipeline CI/CD

L’export JSON généré par l’outil permet d’intégrer l’audit dans un pipeline de test automatisé :

```
- name: Audit Device Integrity
  run: risk-scorer --audit
- name: Check Risk Score
  run: |
```

```

SCORE=$(cat report/audit_report.json | jq .score)
if [ $SCORE -ge 40 ]; then
    echo "DEVICE COMPROMISED"
    exit 1
fi

```

Ce type d'intégration empêche l'exécution de tests sensibles sur des appareils compromis, garantissant l'intégrité des résultats de test.

4.5. Comparaison avec les Outils Existants

Critère	Device Integrity Scorer	In-Risk	SafetyNet Play API	/ Integrity	Drozer	MobSF	RootBeer	Scripts ADB manuels
Langage / Technologie	Python ADB	3.9+	API (cloud)	Google	Python, Java	Python, Docker	Java (bibliothèque Android)	Bash, ADB
Mode d'opération	Passif, observation		Actif, attestation serveur		Actif, analyse IPC	Statique + dynamique	Embarqué dans l'app	Manuel
Connexion réseau requise	Non (hors ligne)		Oui (obligatoire)		Non	Non	Non	Non
Détection root (su, Magisk)	Oui (su + Magisk)		Partielle (contournable)		Non	Partielle	Oui (limitée)	Manuelle
Détection Frida (hook dynamique)	Oui (ps -A)		Non		Non	Partielle	Non	Manuelle
Détection LSPosed / Xposed	Oui		Non		Non	Partielle	Non	Manuelle
Analyse SELinux	Oui		Non		Non	Non	Non	Manuelle
Score de risque quantifié (0–100)	Oui		Non (pass/fail)		Non	Non	Non (booléen)	Non
Réduction des faux positifs	Oui (FP Reducer)		Non		Non	Non	Non	Non
Export JSON / intégration CI/CD	Oui		Non		Non	Oui	Non	Non
Analyse surface d'attaque applicative	Oui (-with-drozer)		Non		Oui	Oui	Non	Non
Contournable par MagiskHide	Difficile		Oui		N/A	N/A	Oui	N/A
Open source	Oui		Non		Oui	Oui	Oui	N/A
Déploiement	CLI installable (pip)		Cloud Google		CLI	Web / Docker	SDK intégré	Terminal

TABLE 5 – Comparaison du Device Integrity Risk Scorer avec les outils existants

5. Conclusions

5.1. Contributions Principales

L'outil apporte plusieurs contributions concrètes au domaine de la sécurité mobile Android :

1. Un pipeline d'audit complet en trois étapes (Collecte → Scoring → Rapport) qui automatise des vérifications qui prendraient plusieurs minutes à réaliser manuellement.
2. Un score normalisé et interprétable (0–100, trois niveaux de risque) qui permet à n'importe quel membre d'une équipe — développeur, testeur, chef de projet — de comprendre le niveau de risque sans expertise en sécurité mobile.

3. Un mécanisme de contextualisation (FP Reducer) qui différencie les environnements de développement légitimes des appareils réellement compromis, réduisant la fatigue d’alerte.
4. Une intégration CI/CD via export JSON qui permet d’automatiser les vérifications d’intégrité dans des workflows DevSecOps.
5. Une approche pédagogique qui explique chaque signal détecté, son risque associé, et les actions de remédiation — transformant l’outil en un instrument de formation autant que d’audit.

5.2. Résultats des Tests

Les scénarios de test ont validé la discrimination du score selon l’environnement : de 5/100 (téléphone physique sain) à 95/100 (émulateur avec Magisk, LSPosed, Frida et SELinux Permissive). L’outil détecte correctement chaque niveau d’ajout de menace et reflète fidèlement l’augmentation du risque dans le score final.

5.3. Assurance Qualité

Métrique	collect/	scorer/	report/	audit.py
Lignes de code	80	60	50	280
Fonctions définies	4	5	2	1
Gestion exceptions	Oui	Non	Non	Non
Timeout ADB (s)	10	N/A	N/A	N/A
Bugs connus	0	0	0	0
Vulnérabilités	0	0	0	0

TABLE 6 – Métriques du code par module

Timeout sur les commandes ADB : Chaque commande ADB est exécutée avec un timeout de 10 secondes. Si l’appareil ne répond pas dans ce délai, la commande retourne une chaîne vide et l’exécution continue sans bloquer l’audit.

Vérification de disponibilité ADB : La fonction `adb_available()` vérifie en amont que l’appareil est connecté et reconnu avant de lancer la collecte. En cas d’échec, l’outil continue en mode dégradé plutôt que de crasher.

Sécurité de l’outil lui-même : L’outil n’exécute que des commandes en lecture seule sur l’appareil — aucune écriture, aucune modification, aucune installation. Les commandes ADB utilisées (`getprop`, `getenforce`, `ps -A`, `pm list packages`) sont toutes des commandes d’observation. Cette conception passive est fondamentale : l’outil est un scanner, non un correcteur.

Reproductibilité : Les résultats de l’outil sont reproductibles à condition que l’état de l’appareil reste identique entre deux exécutions. Le score ne dépend que de l’état observable de l’appareil au moment de l’audit — il est déterministe et ne contient aucun facteur aléatoire.

5.4. *Perspectives d’Amélioration*

- **Base de données de hashes de référence** : Remplacer le hash fixe par une base de données d’empreintes officielles par modèle et version Android.
- **Détection de Frida par méthodes avancées** : Compléter la détection par `ps -A` avec des méthodes basées sur les ports d’écoute et les bibliothèques injectées via `/proc/PID/maps`.
- **Interface graphique** : Un dashboard web généré localement (HTML/CSS) rendrait les rapports plus lisibles et partageables.
- **Mode continu** : Un mode de surveillance en temps réel qui détecte les changements d’état de l’appareil.
- **Intégration Drozer directe** : Automatiser les commandes Drozer dans le pipeline de collecte pour analyser également la surface d’attaque des applications installées.

5.5. *Avertissement Éthique*

L’utilisation du Device Integrity Risk Scorer doit être strictement limitée aux appareils pour lesquels vous disposez d’une autorisation explicite. Cet outil est développé dans un cadre pédagogique et orienté sécurité défensive. L’analyse d’appareils sans consentement est illégale dans la plupart des juridictions. Drozer et Frida ne doivent être utilisés que dans des environnements de test isolés ou avec une autorisation écrite explicite du propriétaire de l’appareil.

Acknowledgements

Nous remercions la communauté open-source derrière Frida, Drozer, et ADB pour les méthodologies qui ont informé la conception de cet outil. Nous exprimons également notre gratitude à ENSA Marrakech pour le soutien apporté à ce projet de recherche en sécurité mobile défensive.

Références

- [1] WithSecureLabs. Drozer — Android Security Assessment Framework. GitHub, 2023.
- [2] NowSecure. Frida — Dynamic Instrumentation Toolkit. GitHub, 2023.

- [3] Google Android Developers. Android Debug Bridge (ADB). Android Developer Documentation.
- [4] topjohnwu. Magisk — The Magic Mask for Android. GitHub, 2023.
- [5] LSPosed Team. LSPosed Framework — ART Hooking Framework. GitHub, 2023.
- [6] Android Open Source Project. Security-Enhanced Linux in Android. Android Developer Documentation.
- [7] Google. Play Integrity API. Android Developer Documentation.
- [8] OWASP Mobile Application Security. OWASP Mobile Application Security Testing Guide (MASTG). OWASP Foundation, 2023.
- [9] Englehardt, S., Narayanan, A. Online Tracking : A 1-million-site Measurement and Analysis. ACM CCS 2016.
- [10] Android Developers. System Properties — getprop/setprop Reference. AOSP Documentation, 2023.
- [11] Bläsing, T., Batyuk, L., Schmidt, A., Camtepe, S., Albayrak, S. An Android Application Sandbox System for Suspicious Software Detection. MALWARE 2010, IEEE.
- [12] Rastogi, V., Chen, Y., Enck, W. AppsPlayground : Automatic Security Analysis of Smartphone Applications. CODASPY 2013, ACM.

Current executable software version

L’outil Device Integrity Risk Scorer est disponible en version exécutable v1.0.0. Les rapports d’audit (JSON et console) générés lors des tests et documentés dans ce rapport sont disponibles dans la branche principale du référentiel GitHub officiel. Les scénarios de test présentés ont été exécutés en mai 2026 sur des émulateurs Genymotion (Android 11) et un téléphone OPPO CPH2239 physique connecté via ADB.